

Frozen Lake from First Principles using Q-Learning

Technical Report - DSCD 614 Reinforcement Learning, Assignment 1

Name: Prince Obeng Nkoah Student ID: 22424742

1. Introduction

This report explains my solution to the 8x8 Frozen Lake problem using Q-Learning. I built everything from scratch in Python and did not use any Reinforcement Learning library (no Gymnasium, OpenAI Gym, Stable Baselines or RLlib). I only used NumPy for the arrays and Matplotlib for the graphs.

In Reinforcement Learning an agent learns by trying actions in an environment. At each step it sees a state, picks an action, and gets a reward and the next state. The goal is to learn a policy (which action to take in each state) that collects the most reward. In Frozen Lake the agent has to move from the Start cell (S) to the Goal cell (G) without falling into a Hole (H). The other cells are Frozen (F) and safe to step on.

2. Environment Design

I wrote the environment as a class called FrozenLakeEnv with the methods the assignment asked for: `reset()`, `step(action)`, `render()`, `get_state()` and `is_terminal()`.

States: I used single numbers from 0 to 63. To get the grid position I do `row = s // 8` and `col = s % 8`. The start is state 0 and the goal is state 63.

Actions: 0 = Left, 1 = Down, 2 = Right, 3 = Up. If the agent tries to move off the grid it just stays where it is.

Rewards: the agent gets +1 only when it reaches the goal, and 0 for every other move (including falling in a hole). Holes and the goal end the episode, and there is also a 200-step limit so an episode always ends.

The 8x8 map used (S start, F frozen, H hole, G goal):

```
S F F F F F F F
F F F F F F F F
F F F H F F F F
F F F H F F F F
F F F H F F F F
F H H F F F H F
F H F F H F H F
F F F H F F F G
```

3. Q-Learning Algorithm

Q-Learning keeps a table $Q(s,a)$ that estimates how good it is to take action a in state s . I start the table at all zeros and update it as the agent learns. The update rule I used is exactly the one from the assignment:

$$Q(s,a) = Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

Here α is the learning rate (how fast it learns), γ is the discount factor (how much it cares about future rewards), and $r + \gamma * \max_{a'} Q(s',a')$ is the target value. When the next state is terminal there is no future value, so that part becomes 0.

For choosing actions I used epsilon-greedy: with probability epsilon the agent picks a random action (explore), otherwise it picks the action with the highest Q-value (exploit). I break ties randomly so it does not always pick action 0. Epsilon starts at 1.0 and slowly goes down to 0.01, so the agent explores a lot at first and then exploits what it learned.

4. Training Methodology

The training loop is in `train.py`. It plays many episodes, updates the Q-table after every step, lowers epsilon after each episode, and saves the episode rewards, successes and epsilon values. The settings I used for the main run were:

Parameter	Value
Episodes	15,000
Learning rate (alpha)	0.1
Discount factor (gamma)	0.99
Initial epsilon	1.0
Minimum epsilon	0.01
Epsilon decay	0.9995
Max steps per episode	200
Random seed	42

I tried a few different values for the learning rate, discount and exploration. $\alpha = 0.1$ with $\gamma = 0.99$ and a slow epsilon decay worked well and learned a good policy. The Q-table and the stats are saved in the `results/` folder.

5. Experimental Results

When I tested the trained agent greedily for 100 episodes on the normal (non-slippery) map it got 100% success, an average reward of 1.0, 0 failures and 100 successful runs. The success rate over the last 1,000 training episodes was about 99.6%. The graphs below show the average reward, success rate and epsilon during training.

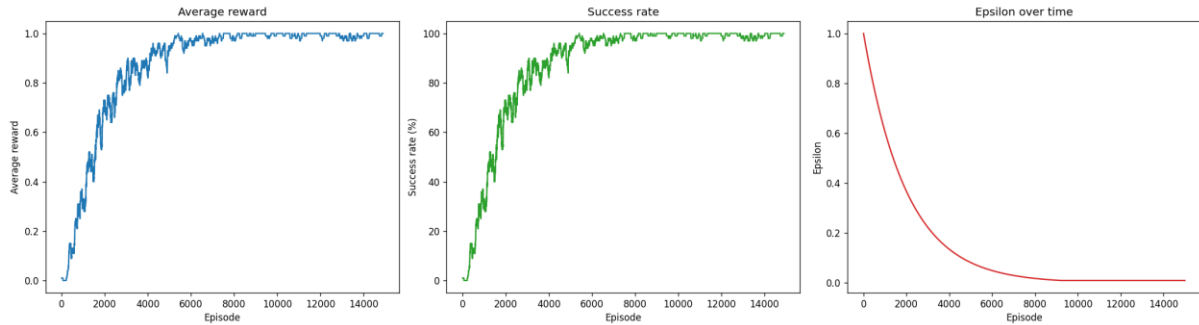


Figure 1: Training performance on the deterministic map (average reward, success rate and epsilon over 15,000 episodes).

Bonus C (strategies): I compared a pure epsilon-greedy (epsilon = 0.1) with a decaying epsilon-greedy. Both reached 100% on the greedy test. The pure one had a slightly higher training success rate (about 92% vs 87%) because it explores less near the end, while the decaying one explores more at the start.

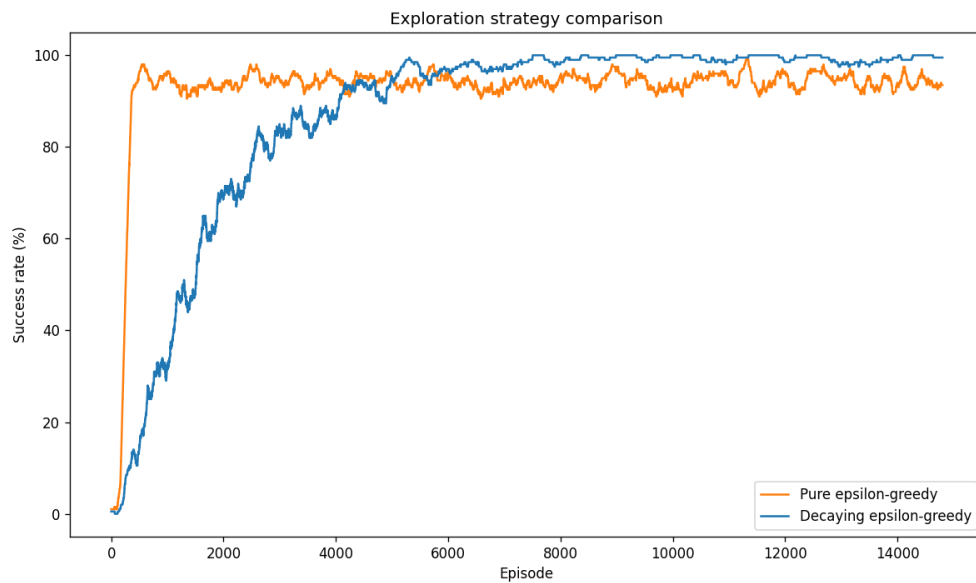


Figure 2: Pure epsilon-greedy vs decaying epsilon-greedy (success rate during training).

Bonus A (slippery): with the slippery version (the action slips about 2/3 of the time) trained for 30,000 episodes, the agent reached 88% success, which is good for a random environment.

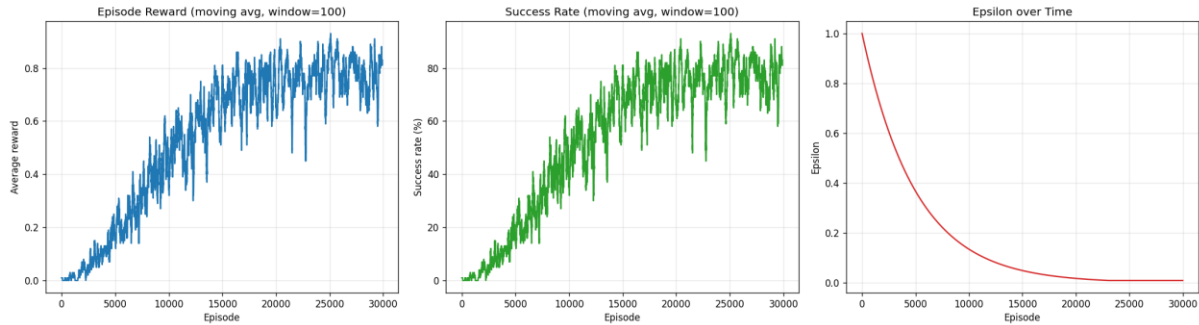


Figure 3: Training performance on the slippery (stochastic) map.

6. Learned Policy

The policy I got from the Q-table (Part D) avoids the holes in the middle and goes down the safe right side to reach the goal:

```

v  v  v  v  v  v  v  v
>  >  >  >  >  v  v  v
^  ^  ^  H  >  >  >  v
^  ^  ^  H  ^  >  >  v
^  ^  ^  H  >  >  >  v
^  H  H  >  ^  ^  H  v
<  H  <  <  H  ^  H  v
<  <  <  H  <  <  <  G

```

(arrows: < Left, v Down, > Right, ^ Up, H Hole, G Goal). Some cells that are not on the best path show random arrows because the agent never visits them when it plays greedily from the start.

7. Challenges Encountered

- Reward only at the goal: because the reward only comes at the end, the agent fails a lot early on. I needed enough episodes and a slow epsilon decay for the values to spread back through the table.
- Ties in the Q-table: at the start all the values are 0, so every action is a tie. Picking randomly between tied actions (instead of always action 0) helped the agent explore better.
- Slippery mode: the random slipping made learning much harder and needed about twice as many episodes.
- Printing arrows: I had to set the output to UTF-8 so the arrows showed up correctly on Windows.

8. Conclusion

My from-scratch Q-Learning agent solves the normal 8x8 Frozen Lake perfectly (100% success) and still does well on the slippery version (88%). I split the code into the environment, the agent, training and evaluation, and I did all three bonus tasks. The results show the main ideas of Q-

Learning: updating the Q-table with the TD rule and lowering epsilon over time lets the agent go from exploring to exploiting and find a good, hole-avoiding policy.